

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №3**  
**по дисциплине «Построение и анализ алгоритмов»**  
**ТЕМА: РЕДАКЦИОННОЕ РАССТОЯНИЕ.**

Студент гр. 4343

Селезнев Д. Е.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

## Цель работы.

Изучить методы вычисления редакционного расстояния между строками. Реализовать на языке программирования Python алгоритм Вагнера – Фишера для нахождения минимальной стоимости преобразования одной строки в другую с учётом заданных цен операций (вставка, удаление, замена), а также алгоритм восстановления оптимального редакционного предписания. Реализовать вычисление расстояния Левенштейна как частного случая задачи. Дополнительно разработать модификацию алгоритма с учётом особых условий (изменённой стоимости замены на заданный символ и удаления определённого символа). Провести тестирование реализованных алгоритмов на различных входных данных и проверить корректность полученных результатов.

## Задание.

### Задание №1

Над строкой  $\varepsilon$  (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1.  $replace(\varepsilon, a, b)$  – заменить символ  $a$  на символ  $b$ .
2.  $insert(\varepsilon, a)$  – вставить в строку символ  $a$  (на любую позицию).
3.  $delete(\varepsilon, b)$  – удалить из строки символ  $b$ .

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки  $A$  и  $B$ , а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки  $A$  в строку  $B$ .

Входные данные: первая строка – три числа: цена операции  $replace$ , цена операции  $insert$ , цена операции  $delete$ ; вторая строка –  $A$ ; третья строка –  $B$ .

Выходные данные: одно число – минимальная стоимость операций.

### Задание №2

Над строкой  $\varepsilon$  (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1.  $replace(\epsilon, a, b)$  – заменить символ  $a$  на символ  $b$ .
2.  $insert(\epsilon, a)$  – вставить в строку символ  $a$  (на любую позицию).
3.  $delete(\epsilon, b)$  – удалить из строки символ  $b$ .

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки  $A$  и  $B$ , а также три числа, отвечающие за цену каждой операции. Определите последовательность операций (редакционное предписание) с минимальной стоимостью, которые необходимы для превращения строки  $A$  в строку  $B$ .

Входные данные: первая строка – три числа: цена операции  $replace$ , цена операции  $insert$ , цена операции  $delete$ ; вторая строка –  $A$ ; третья строка –  $B$ .

Выходные данные: первая строка – последовательность операций ( $M$  – совпадение, ничего делать не надо;  $R$  – заменить символ на другой;  $I$  – вставить символ на текущую позицию;  $D$  – удалить символ из строки); вторая строка – исходная строка  $A$ ; третья строка – исходная строка  $B$ .

### **Задание №3**

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Первая строка входных данных содержит строку из строчных латинских букв. ( $S, 1 \leq |S| \leq 2550$ ).

Вторая строка входных данных содержит строку из строчных латинских букв. ( $T, 1 \leq |T| \leq 2550$ ).

Параметры выходных данных:

Одно число  $L$ , равное расстоянию Левенштейна между строками  $S$  и  $T$ .

### **Вариант 2**

Вар. 2. "Особый заменитель и особо удаляемый символ": цена замены на определённый символ отличается от обычной цены замены; цена удаления

другого (или того же) определённого символа отличается от обычной цены удаления. Особый заменитель и цена замены на него, особо удаляемый символ и цена его удаления – дополнительные входные данные.

### **Теоретический минимум.**

Редакционное расстояние – это мера различия между двумя строками, определяемая как минимальная стоимость последовательности операций, необходимых для преобразования одной строки в другую. К допустимым операциям относятся:

- вставка символа (insert),
- удаление символа (delete),
- замена символа (replace).

Каждой операции может быть назначена определённая стоимость.

В данной лабораторной работе рассматриваются:

- алгоритм Вагнера – Фишера для нахождения минимальной стоимости преобразования;
- восстановление редакционного предписания;
- расстояние Левенштейна как частный случай;
- модифицированная задача с особыми стоимостями операций.

### **Алгоритм Вагнера – Фишера**

Алгоритм Вагнера – Фишера основан на методе динамического программирования и позволяет вычислить минимальную стоимость преобразования строки  $A$  в строку  $B$ .

Основная идея работы алгоритма:

- 1) Строится матрица  $D$  размера  $(|A| + 1) \times (|B| + 1)$ , где элемент  $D[i][j]$  – минимальная стоимость преобразования первых  $i$  символов строки  $A$  в первые  $j$  символов строки  $B$ .
- 2) Инициализация:
  - $D[0][j] = j \cdot cost_{insert}$  – удаление всех символов строки  $A$ ,
  - $D[i][0] = i \cdot cost_{delete}$  – вставка всех символов строки  $B$ .

3) Для остальных элементов используется рекуррентное соотношение:

- если  $A[i] = B[j]$ , то стоимость не увеличивается;
- иначе выбирается минимум из трёх операций: вставка, удаление или замена.

4) Значение в правом нижнем углу матрицы даёт итоговую минимальную стоимость.

### **Восстановление редакционного предписания**

Помимо вычисления стоимости, алгоритм позволяет восстановить последовательность операций.

Основная идея:

- 1) После заполнения матрицы выполняется обратный проход от ячейки  $D[n][m]$  к  $D[0][0]$ .
- 2) На каждом шаге определяется, какая операция привела к текущему значению:
  - если символы совпадают – операция M (match),
  - если переход был из  $(i - 1, j - 1)$  – R (replace),
  - если переход был из  $(i, j - 1)$  – I (insert),
  - если переход был из  $(i - 1, j)$  – D (delete),
- 3) Последовательность операций формируется в обратном порядке и затем разворачивается.

### **Расстояние Левенштейна**

Расстояние Левенштейна – частный случай редакционного расстояния, в котором стоимость всех операций равна 1.

В реализации используется оптимизация по памяти: вместо полной матрицы хранятся только две строки предыдущая (*prev*) и текущая (*curr*). Это позволяет снизить потребление памяти с  $O(n \cdot m)$  до  $O(m)$ .

### **Модифицированная задача**

В расширенной постановке учитываются дополнительные условия:

- замена на определённый символ имеет особую стоимость;

- удаление определённого символа имеет отличную стоимость.

Основная идея:

- 1) При вычислении переходов в динамической таблице проверяется:
  - является ли символ «особым»;
  - применяется ли специальная стоимость операции.
- 2) Алгоритм сохраняет ту же структуру, но учитывает дополнительные условия при выборе минимума.

### Временная сложность

- Алгоритм Вагнера–Фишера: заполнение матрицы размером  $n \times m$  –  $O(n \cdot m)$
- Восстановление ответа: обратный проход по матрице  $O(n + m)$
- Расстояние Левенштейна: аналогично  $O(n \cdot m)$
- Модифицированный алгоритм:  $O(n \cdot m)$

### Пространственная сложность

- Хранение матрицы:  $O(n \cdot m)$
- Оптимизированный алгоритм Левенштейна:  $O(m)$

### Выполнение работы.

#### 1. Алгоритм Вагнера–Фишера

Программа реализует алгоритм Вагнера – Фишера для нахождения минимальной стоимости преобразования одной строки в другую. Алгоритм использует метод динамического программирования и гарантирует получение оптимального решения.

*wagner\_fischer(replace\_cost, insert\_cost, delete\_cost, A, B)* – функция вычисления минимальной стоимости преобразования строки  $A$  в строку  $B$ . В начале определяется длина строк и создаётся двумерный массив  $dp$  размера  $(n + 1) \times (m + 1)$ , где каждый элемент соответствует минимальной стоимости преобразования. Далее выполняется инициализация: первый столбец заполняется значениями удаления символов строки  $A$ , первая строка – вставки символов строки  $B$ . Далее двойной цикл обрабатывает все пары префиксов:

- если  $A[i - 1] = B[j - 1]$ , то

$$dp[i][j] = dp[i - 1][j - 1]$$

- иначе:

$$dp[i][j] = \min \begin{cases} dp[i - 1][j - 1] + \text{replace\_cost} \\ dp[i][j - 1] + \text{insert\_cost} \\ dp[i - 1][j] + \text{delete\_cost} \end{cases}$$

Итоговый ответ находится в ячейке  $dp[n][m]$ .

### Пайплайн

В блоке `if __name__ == "__main__":` программа считывает три значения стоимостей операций (замена, вставка, удаление), а также строки  $A$  и  $B$ . Предобработка входных данных не требуется, так как строки используются напрямую. Далее вызывается функция `wagner_fischer(...)`. Постобработка отсутствует, так как результат уже содержится в итоговой ячейке матрицы. Вывод осуществляется с помощью `print` – на экран выводится минимальная стоимость преобразования.

### Восстановление редакционного предписания

Программа расширяет алгоритм Вагнера-Фишера, добавляя возможность восстановления последовательности операций, необходимых для преобразования одной строки в другую.

`wagner_fischer_with_path(replace_cost, insert_cost, delete_cost, A, B)` – функция, восстанавливающая последовательность операций преобразования. Сначала полностью строится матрица  $dp$  аналогично базовому алгоритму. Далее инициализируется пустой список `operations = []` для хранения операций и устанавливаются указатели  $i, j = n, m$  в правый нижний угол таблицы. Цикл выполняется до достижения левого верхнего угла. Сначала обрабатываются граничные случаи: если  $i == 0$  (достигнут верхний край таблицы), остались только операции вставки, добавляется символ  $I$  в список `operations` и декрементируется  $j$ ; если  $j == 0$  (достигнут левый край), остались только операции удаления, добавляется  $D$  и декрементируется  $i$ . Для основных случаев проверяется совпадение символов: если  $A[i - 1] ==$

$B[j - 1]$  , символы совпадают, добавляется  $M$  , и оба указателя декрементируются. В противном случае вычисляются стоимости трёх возможных предыдущих состояний, находится минимальная стоимость, и выбирается соответствующая операция. Если  $min\_val == insert\_val$  , добавляется  $I$  и декрементируется  $j$  ; иначе если  $min\_val == delete\_val$  , добавляется  $D$  и декрементируется  $i$  ; иначе добавляется  $R$  (*replace*) и декрементируются оба указателя. После завершения цикла список операций переворачивается, так как путь восстанавливался в обратном порядке от конца к началу.

### Пайплайн

В блоке `if __name__ == "__main__":` программа считывает стоимости операций и строки  $A$  и  $B$  . Далее вызывается функция `wagner_fischer_with_path(...)`. Постобработка включает разворот полученной последовательности. Вывод осуществляется с помощью `print`: сначала выводится строка операций, затем исходные строки  $A$  и  $B$ .

### Алгоритм Левенштейна

Программа реализует классический алгоритм вычисления расстояния Левенштейна с оптимизацией по памяти.

`levenshtein_distance(S, T)` – вычисляет минимальное редакционное расстояние между строками  $S$  и  $T$ , используя три базовые операции: вставку, удаление и замену символа (каждая стоимостью 1). Функция инициализирует переменные  $n = len(S)$  и  $m = len(T)$  , создаёт массив `prev = list(range(m + 1))` , представляющий предыдущую строку таблицы динамического программирования со значениями  $[0, 1, 2, \dots, m]$  , и массив `curr = [0] * (m + 1)` для текущей строки. Внешний цикл проходит по символам строки  $S$ , устанавливая базовое значение `curr[0] = i`. Внутренний проходит по символам строки  $T$ . Если символы совпадают, значение берётся из диагонали: `curr[j] = prev[j - 1]`. Иначе вычисляется `curr[j] = 1 + min(prev[j - 1], curr[j - 1], prev[j])` , где аргументы соответствуют операциям замены, вставки и удаления. После обработки строки выполняется



обмен  $prev, curr = curr, prev$  . Функция возвращает  $prev[m]$  – минимальное расстояние между строками.

### Пайплайн

В блоке `if __name__ == "__main__"`: программа считывает две строки  $S$  и  $T$  . Предобработка отсутствует. Далее вызывается функция  $levenshtein\_distance(S, T)$ . Постобработка не требуется. Вывод осуществляется с помощью `print` – выводится расстояние Левенштейна.

### Алгоритм Вагнера-Фишера со специальными символами

Программа реализует модифицированный алгоритм Вагнера – Фишера с поддержкой специальных символов, имеющих отличную от стандартной стоимость операций.

$wagner\_fischer\_special(A, B, replace\_cost, insert\_cost, delete\_cost, special\_replace\_char, special\_replace\_cost, special\_delete\_char, special\_delete\_cost)$  – вычисляет минимальное редакционное расстояние с учётом специальных стоимостей для определённых символов. Инициализация и создание таблицы  $dp$  аналогичны стандартному алгоритму. Ключевое отличие заключается в заполнении базовых случаев и основной таблицы: при заполнении первого столбца для каждого символа  $A[i - 1]$  проверяется, является ли он специальным символом удаления, и в зависимости от этого используется либо  $special\_delete\_cost$ , либо стандартный  $delete\_cost$ . При заполнении основной таблицы для несовпадающих символов динамически определяются текущие стоимости операций: если  $B[j - 1] == special\_replace\_char$ , используется  $special\_replace\_cost$  вместо  $replace\_cost$ ; если  $A[i - 1] == special\_delete\_char$ , используется  $special\_delete\_cost$  вместо  $delete\_cost$ . Стоимость вставки остаётся стандартной. Функция возвращает  $dp[n][m]$ .

### Пайплайн

В блоке `if __name__ == "__main__"`: программа считывает стоимости операций, строки  $A$  и  $B$ , а также дополнительные параметры: специальный символ замены и его стоимость, специальный символ удаления и его

стоимость. Предобработка заключается в интерпретации этих параметров для дальнейшего использования в алгоритме. Далее вызывается функция *wagner\_fischer\_special(...)*. Постобработка отсутствует. Вывод осуществляется с помощью *print* – выводится минимальная стоимость преобразования.

### **Тестирование.**

#### **Алгоритм Вагнера–Фишера**

Для проверки корректности реализации алгоритма Вагнера–Фишера был разработан набор модульных тестов с использованием фреймворка *pytest*. В процессе тестирования использовались как базовые тестовые случаи, так и реальные примеры, взятые из системы *Stepik*, что позволяет дополнительно подтвердить корректность работы алгоритма на практических данных. Тестирование включает проверку граничных случаев, таких как работа с пустыми строками и преобразование строки в пустую и обратно. Также проверяется случай совпадающих строк, при котором стоимость преобразования должна быть равна нулю. Отдельное внимание уделяется стандартным случаям с равными стоимостями операций, а также ситуациям с изменёнными стоимостями, влияющими на итоговый результат. Дополнительно используется реальный пример из *Stepik*, который подтверждает корректность работы алгоритма на нестандартных входных данных.

#### **Восстановление редакционного предписания**

Для проверки корректности реализации восстановления последовательности операций был разработан набор модульных тестов с использованием *pytest*. Тестирование включает проверку граничных случаев с пустыми строками, при которых формируются только операции вставки или удаления, а также случай совпадающих строк, где результат состоит исключительно из операций совпадения. Дополнительно проверяются простые преобразования с одной операцией, такие как замена и вставка символов, с контролем корректного состава операций и количества совпадений. Также проводится проверка корректности структуры результата на

стандартном примере преобразования строк, включая соответствие допустимым операциям.

### **Алгоритм Левенштейна**

Для проверки корректности реализации алгоритма Левенштейна был разработан набор модульных тестов с использованием *pytest*. Тестирование включает проверку граничных случаев, таких как работа с пустыми строками и совпадающими строками. Также рассматриваются случаи, в которых требуется выполнение одной операции, например вставки, удаления или замены символа. Дополнительно проверяются более сложные случаи, требующие нескольких операций преобразования. Корректность реализации подтверждается совпадением полученных результатов с известными эталонными значениями для классических тестовых примеров.

### **Модифицированный алгоритм**

Для проверки корректности реализации модифицированного алгоритма Вагнера–Фишера был разработан набор модульных тестов с использованием фреймворка *pytest*. Тестирование направлено на проверку корректности вычисления редакционного расстояния при наличии специальных стоимостей операций. В частности, рассматриваются случаи, в которых специальные символы для замены и удаления отсутствуют во входных строках, что должно приводить к работе алгоритма как в базовой версии. Также проверяются ситуации, в которых стоимость специальной замены отличается от обычной, что влияет на выбор оптимальной операции преобразования. Отдельное внимание уделяется проверке работы специального символа удаления, включая случаи, когда его стоимость выше или ниже стандартной стоимости удаления. Дополнительно тестируются случаи с повторяющимися символами и совпадающими строками, содержащими специальные символы, при которых стоимость преобразования должна быть равна нулю.

Программный код реализации тестов см. в приложении А.

Результаты тестирования см. в приложении Б.

## **Вывод.**

В ходе выполнения лабораторной работы были реализованы алгоритм Вагнера–Фишера для вычисления редакционного расстояния между строками, алгоритм восстановления последовательности редакционных операций, а также оптимизированная версия алгоритма Левенштейна и модифицированный вариант с учётом специальных стоимостей операций.

Алгоритм Вагнера–Фишера позволяет находить минимальную стоимость преобразования одной строки в другую с использованием динамического программирования. Реализация основана на построении матрицы, в которой последовательно вычисляются минимальные стоимости вставки, удаления и замены символов с учётом заданных весов операций.

Дополнительно реализован алгоритм восстановления редакционного предписания, который позволяет получить последовательность операций, приводящих одну строку к другой с минимальной стоимостью. Восстановление выполняется путём обратного прохода по матрице динамического программирования и выбора соответствующих операций преобразования.

Реализация алгоритма Левенштейна основана на оптимизации по памяти за счёт хранения только двух строк матрицы, что позволяет снизить пространственную сложность.

Также реализован модифицированный алгоритм, учитывающий специальные стоимости замены и удаления отдельных символов, что расширяет классическую постановку задачи.

Корректность реализованных алгоритмов подтверждена с помощью набора модульных тестов, проверяющих граничные случаи, стандартные ситуации и влияние различных стоимостей операций на итоговый результат. Дополнительно проверяется корректность восстановления последовательности операций и соответствие результатов эталонным значениям.

Таким образом, в ходе работы были изучены и реализованы основные методы решения задачи редакционного расстояния, исследованы их модификации и особенности, а также подтверждена корректность их работы на тестовых данных.

## ПРИЛОЖЕНИЕ А

### ИСХОДНЫЙ КОД ПРОГРАММЫ

#### **wagner\_fischer.py**

```
def wagner_fischer(replace_cost, insert_cost, delete_cost, A, B):
    n = len(A)
    m = len(B)

    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = i * delete_cost

    for j in range(m + 1):
        dp[0][j] = j * insert_cost

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if A[i - 1] == B[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = min(
                    dp[i - 1][j - 1] + replace_cost,
                    dp[i][j - 1] + insert_cost,
                    dp[i - 1][j] + delete_cost
                )

    return dp[n][m]

if __name__ == "__main__":
    costs = list(map(int, input().split()))
    replace_cost = costs[0]
    insert_cost = costs[1]
    delete_cost = costs[2]

    A = input().strip()
    B = input().strip()

    print(edit_distance(replace_cost, insert_cost, delete_cost, A, B))
```

#### **wagner\_fischer\_path.py**

```
def wagner_fischer_with_path(replace_cost, insert_cost, delete_cost, A,
B):
    n = len(A)
    m = len(B)

    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = i * delete_cost

    for j in range(m + 1):
        dp[0][j] = j * insert_cost

    for i in range(1, n + 1):
        for j in range(1, m + 1):
```

```

        if A[i - 1] == B[j - 1]:
            dp[i][j] = dp[i - 1][j - 1]
        else:
            dp[i][j] = min(
                dp[i - 1][j - 1] + replace_cost,
                dp[i][j - 1] + insert_cost,
                dp[i - 1][j] + delete_cost
            )

operations = []
i, j = n, m

while i > 0 or j > 0:
    if i == 0:
        operations.append('I')
        j -= 1
    elif j == 0:
        operations.append('D')
        i -= 1
    elif A[i - 1] == B[j - 1]:
        operations.append('M')
        i -= 1
        j -= 1
    else:
        replace_val = dp[i - 1][j - 1] + replace_cost
        insert_val = dp[i][j - 1] + insert_cost
        delete_val = dp[i - 1][j] + delete_cost

        min_val = min(replace_val, insert_val, delete_val)

        if min_val == insert_val:
            operations.append('I')
            j -= 1
        elif min_val == delete_val:
            operations.append('D')
            i -= 1
        else:
            operations.append('R')
            i -= 1
            j -= 1

operations = operations[::-1]

return ''.join(operations)

if __name__ == "__main__":
    costs = list(map(int, input().split()))
    replace_cost = costs[0]
    insert_cost = costs[1]
    delete_cost = costs[2]

    A = input().strip()
    B = input().strip()

    path = wagner_fischer_with_path(replace_cost, insert_cost,
delete_cost, A, B)
    print(path)
    print(A)

```

```
print(B)
```

### **levenshtein.py**

```
def levenshtein_distance(S, T):
    n = len(S)
    m = len(T)

    prev = list(range(m + 1))
    curr = [0] * (m + 1)

    for i in range(1, n + 1):
        curr[0] = i
        char_a = S[i - 1]

        for j in range(1, m + 1):
            char_b = T[j - 1]

            if char_a == char_b:
                curr[j] = prev[j - 1]
            else:
                curr[j] = 1 + min(prev[j - 1], curr[j - 1], prev[j])

        prev, curr = curr, prev

    return prev[m]

if __name__ == "__main__":
    S = input()
    T = input()

    print(levenshtein_distance(S, T))
```

### **wagner\_fischer\_special.py**

```
def wagner_fischer_special(A, B, replace_cost, insert_cost, delete_cost,
                           special_replace_char, special_replace_cost,
                           special_delete_char, special_delete_cost):
    n = len(A)
    m = len(B)

    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        if A[i - 1] == special_delete_char:
            dp[i][0] = dp[i - 1][0] + special_delete_cost
        else:
            dp[i][0] = dp[i - 1][0] + delete_cost

    for j in range(1, m + 1):
        dp[0][j] = dp[0][j - 1] + insert_cost

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if A[i - 1] == B[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                if B[j - 1] == special_replace_char:
                    current_replace_cost = special_replace_cost
```



```

        else:
            current_replace_cost = replace_cost

        if A[i - 1] == special_delete_char:
            current_delete_cost = special_delete_cost
        else:
            current_delete_cost = delete_cost

        dp[i][j] = min(
            dp[i - 1][j - 1] + current_replace_cost,
            dp[i][j - 1] + insert_cost,
            dp[i - 1][j] + current_delete_cost
        )

    return dp[n][m]

if __name__ == "__main__":
    costs = list(map(int, input().split()))
    replace_cost = costs[0]
    insert_cost = costs[1]
    delete_cost = costs[2]

    A = input().strip()
    B = input().strip()

    special_replace_char, special_replace_cost = input().split()
    special_replace_cost = int(special_replace_cost)

    special_delete_char, special_delete_cost = input().split()
    special_delete_cost = int(special_delete_cost)

    result = wagner_fischer_special(
        A, B,
        replace_cost, insert_cost, delete_cost,
        special_replace_char, special_replace_cost,
        special_delete_char, special_delete_cost
    )

    print(result)

```

### **test\_wagner\_fischer.py**

```

import pytest
from wagner_fischer import wagner_fischer

def test_empty_strings():
    assert wagner_fischer(1, 1, 1, "", "") == 0
    assert wagner_fischer(1, 1, 1, "abc", "") == 3
    assert wagner_fischer(1, 1, 1, "", "abc") == 3

def test_identical_strings():
    assert wagner_fischer(1, 1, 1, "hello", "hello") == 0

def test_standard_costs():
    assert wagner_fischer(1, 1, 1, "kitten", "sitting") == 3

def test_high_costs():
    assert wagner_fischer(1, 10, 1, "", "abc") == 30
    assert wagner_fischer(1, 1, 10, "abc", "") == 30

```

## **test\_wagner\_fischer\_path.py**

```
import pytest
from wagner_fischer_path import wagner_fischer_with_path

def test_empty_strings():
    assert wagner_fischer_with_path(1, 1, 1, "", "") == ""
    assert wagner_fischer_with_path(1, 1, 1, "abc", "") == "DDD"
    assert wagner_fischer_with_path(1, 1, 1, "", "abc") == "III"

def test_identical_strings():
    result = wagner_fischer_with_path(1, 1, 1, "hello", "hello")
    assert result == "MMMMM"

def test_single_operations():
    result = wagner_fischer_with_path(1, 1, 1, "cat", "bat")
    assert 'R' in result and result.count('M') == 2

    result = wagner_fischer_with_path(1, 1, 1, "cat", "cats")
    assert 'I' in result and result.count('M') == 3

def test_path_correctness():
    result = wagner_fischer_with_path(1, 1, 1, "kitten", "sitting")
    assert len(result) == 7
    assert all(op in 'MRID' for op in result)
```

## **test\_levenshtein.py**

```
import pytest
from levenshtein import levenshtein_distance

def test_empty_strings():
    assert levenshtein_distance("", "") == 0
    assert levenshtein_distance("abc", "") == 3
    assert levenshtein_distance("", "abc") == 3

def test_identical_strings():
    assert levenshtein_distance("hello", "hello") == 0

def test_single_operation():
    assert levenshtein_distance("cat", "bat") == 1
    assert levenshtein_distance("cat", "cats") == 1
    assert levenshtein_distance("cats", "cat") == 1

def test_multiple_operations():
    assert levenshtein_distance("kitten", "sitting") == 3
    assert levenshtein_distance("saturday", "sunday") == 3
```

## **test\_wagner\_fischer\_special.py**

```
import pytest
from wagner_fischer_special import wagner_fischer_special

def test_no_special_chars():
    result = wagner_fischer_special(
        "abc", "xyz",
        replace_cost=1, insert_cost=1, delete_cost=1,
        special_replace_char='@', special_replace_cost=2,
        special_delete_char='#', special_delete_cost=2
```

```

    )
    assert result == 3

def test_special_replace_char():
    # Высокая стоимость специальной замены
    result = wagner_fischer_special(
        "a", "@",
        replace_cost=1, insert_cost=1, delete_cost=1,
        special_replace_char='@', special_replace_cost=2,
        special_delete_char='#', special_delete_cost=1
    )
    assert result == 2 # замена или удаление + вставка (одинаково)

    # Низкая стоимость специальной замены
    result2 = wagner_fischer_special(
        "a", "@",
        replace_cost=5, insert_cost=5, delete_cost=5,
        special_replace_char='@', special_replace_cost=1,
        special_delete_char='#', special_delete_cost=5
    )
    assert result2 == 1 # специальная замена дешевле

def test_special_delete_char():
    # Обычное удаление
    result1 = wagner_fischer_special(
        "ab", "b",
        replace_cost=2, insert_cost=2, delete_cost=1,
        special_replace_char='@', special_replace_cost=2,
        special_delete_char='#', special_delete_cost=3
    )
    assert result1 == 1 # обычное удаление

    # Специальное удаление (дороже обычного)
    result2 = wagner_fischer_special(
        "#b", "b",
        replace_cost=2, insert_cost=2, delete_cost=1,
        special_replace_char='@', special_replace_cost=2,
        special_delete_char='#', special_delete_cost=3
    )
    assert result2 == 3 # специальное удаление

def test_cheap_special_delete():
    result = wagner_fischer_special(
        "###", "",
        replace_cost=5, insert_cost=5, delete_cost=5,
        special_replace_char='@', special_replace_cost=5,
        special_delete_char='#', special_delete_cost=1
    )
    assert result == 3

def test_identical_strings_with_special():
    result = wagner_fischer_special(
        "#@#", "#@#",
        replace_cost=1, insert_cost=1, delete_cost=1,
        special_replace_char='@', special_replace_cost=2,
        special_delete_char='#', special_delete_cost=2
    )
    assert result == 0

```

## ПРИЛОЖЕНИЕ Б

### ТЕСТИРОВАНИЕ

Таблица 1 - Результаты тестирования алгоритма Вагнера – Фишера

| № п/п | Входные данные                              | Выходные данные | Комментарии                                            |
|-------|---------------------------------------------|-----------------|--------------------------------------------------------|
| 1.    | ("", ""), (abc, ""), ("", abc)              | 0, 3, 3         | Корректная работа граничных случаев с пустыми строками |
| 2.    | ("hello", "hello")                          | 0               | Проверка совпадающих строк                             |
| 3.    | ("kitten", "sitting")                       | 3               | Корректная работа стандартного случая                  |
| 4.    | ("entrance", "reenterable")                 | 5               | Проверка на реальном примере преобразования строк      |
| 5.    | (replace=1, insert=10, delete=1, "", "abc") | 30              | Проверка влияния высоких стоимостей операций вставки   |
| 6.    | (replace=1, insert=1, delete=10, "abc", "") | 30              | Проверка влияния высоких стоимостей операций удаления  |

Таблица 2 - Результаты тестирования восстановления редакционного предписания

| № п/п | Входные данные        | Выходные данные        | Комментарии                                            |
|-------|-----------------------|------------------------|--------------------------------------------------------|
| 1.    | ("", "")              | ""                     | Корректная работа граничного случая с пустыми строками |
| 2.    | ("abc", "")           | "DDD"                  | Проверка последовательности удалений                   |
| 3.    | ("", "abc")           | "III"                  | Проверка последовательности вставок                    |
| 4.    | ("hello", "hello")    | "MMMMM"                | Корректная работа при совпадающих строках              |
| 5.    | ("cat", "bat")        | содержит R, M=2        | Проверка операции замены                               |
| 6.    | ("cat", "cats")       | содержит I, M=3        | Проверка операции вставки                              |
| 7.    | ("kitten", "sitting") | длина 7, операции MRID | Проверка корректности последовательности операций      |

Таблица 3 - Результаты тестирования модифицированного алгоритма

| № п/п | Входные данные | Выходные данные | Комментарии                                                |
|-------|----------------|-----------------|------------------------------------------------------------|
| 1.    | ("abc", "xyz") | 3               | Корректная работа базового случая без специальных символов |

|    |                                                                                                                                                                     |   |                                                                     |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---------------------------------------------------------------------|
| 2. | A = "a", B="@", replace=1,<br>insert=1, delete=1,<br>special_replace_char='@',<br>special_replace_cost=2,<br>special_delete_char='#',<br>special_delete_cost=1      | 2 | Специальная замена не даёт выигрыша                                 |
| 3. | A = "a", B="@", replace=5,<br>insert=5, delete=5,<br>special_replace_char='@',<br>special_replace_cost=1,<br>special_delete_char='#',<br>special_delete_cost=5      | 1 | Специальная замена выгоднее стандартных операций                    |
| 4. | ("ab", "b")                                                                                                                                                         | 1 | Корректная работа обычного удаления символа                         |
| 5. | A="#b", B="b", replace=2,<br>insert=2, delete=1,<br>special_replace_char='@',<br>special_replace_cost=2,<br>special_delete_char='#',<br>special_delete_cost=3       | 3 | Специальное удаление символа                                        |
| 6. | A="###", B="", replace=5,<br>insert=5, delete=5,<br>special_replace_char='@',<br>special_replace_cost=5,<br>special_delete_char='#',<br>special_delete_cost=1       | 3 | Множественное удаление специальных символов                         |
| 7. | A="#@#", B="#@#",<br>replace=1, insert=1,<br>delete=1,<br>special_replace_char='@',<br>special_replace_cost=2,<br>special_delete_char='#',<br>special_delete_cost=2 | 0 | Корректная работа при совпадающих строках со специальными символами |

Таблица 4 - Результаты тестирования алгоритма Левенштейна

| № п/п | Входные данные                 | Выходные данные | Комментарии                                            |
|-------|--------------------------------|-----------------|--------------------------------------------------------|
| 1.    | ("", ""), (abc, ""), ("", abc) | 0, 3, 3         | Корректная работа граничных случаев с пустыми строками |
| 2.    | ("hello", "hello")             | 1               | Проверка совпадающих строк                             |
| 3.    | ("cat", "bat")                 | 1               | Проверка одной операции замены                         |
| 4.    | ("cat", "cats")                | 1               | Проверка вставки символа                               |
| 5.    | ("cats", "cat")                | 1               | Проверка удаления символа                              |
| 6.    | ("kitten", "sitting")          | 3               | Классический пример Левенштейна                        |
| 7.    | ("saturday", "sunday")         | 3               | Сложный случай с несколькими операциями                |

